

## LABORATORY 10

Classification, Clustering, Evaluation, and Local Deployment

Course: Data Analytics and Scalability- Laboratory

Group Work (Execution) – Individual Submission (Assessment)

Models: Logistic Regression (Classification) and K-Means

Clustering Dataset: Same dataset used across tasks

Duration: 3 Hours | Total Points: 60

### TASK 1 — Data Preparation

Load and inspect the dataset.

The screenshot displays the Anaconda Jupyter Notebook environment. The left sidebar shows a file explorer with various datasets and notebooks. The main area contains a code cell with the following Python code:

```
[12]: print("--- Data Inspection ---")
print(df.head().to_markdown(index=False))
df.info()
```

The output of the code is displayed below the cell:

```
--- Data Inspection ---
| park_id | month | entry_fee | marketing_spend | avg_temp_c | rainy_days |
| near_city | trail_km_open | monthly_visitors | high_demand |
|-----|-----|-----|-----|-----|-----|
| 623 | Jan | 181.703 | 35745.1 | 22.8468 | 21 |
| no | 1486 | Jan | 67.4533 | 2040.45 | 1 |
| yes | 13 | May | 21.3198 | 29319.2 | 19.2204 | 4 |
| yes | 167 | May | 232.114 | 4704.03 | 1 |
| yes | 13 | Aug | 13.9421 | 14675.4 | 23.2835 | 4 |
| no | 1010 | Oct | 240.886 | 1004.91 | 0 |
| no | 13 | Aug | 19.4794 | 4969.38 | 24.7722 | 19 |
| no | 1010 | Oct | 182.299 | 1903.34 | 0 |
| no | 13 | Aug | 19.5654 | 63247.2 | 26.7125 | 8 |
| no | 13 | Aug | 19.5654 | 927.117 | 0 |

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21685 entries, 0 to 21684
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   park_id         21685 non-null  int64
1   month           21685 non-null  object
2   entry_fee       21685 non-null  float64
3   marketing_spend 21685 non-null  float64
4   avg_temp_c      21685 non-null  float64
5   rainy_days      21685 non-null  int64
6   near_city       21685 non-null  object
7   trail_km_open   21685 non-null  float64
8   monthly_visitors 21685 non-null  float64
9   high_demand     21685 non-null  int64
```

The right sidebar features the Anaconda Assistant chat interface, which includes a "Get started" section with links to "Load volcano data from the Smithsonian Institute", "Plot the first 100 Fibonacci numbers", and "Create a function to mask emails". Below this is a "Working with dataframes" section with a link to "Load a DataFrame". At the bottom, there is a "Start chat without notebook context..." button and a "Attach DataFrame" button.

## Apply common preparation steps.

The screenshot displays a Jupyter Notebook environment within a web browser. The notebook is titled "Apply common preparation steps." and contains two code cells. The first cell calculates the median of the 'monthly\_visitors' column and creates a new column 'high\_demand' based on this median. The second cell defines categorical and numerical columns, creates a feature matrix X and target vector y, and prints the total rows and target distribution.

```
[13]: median_visitors = df['monthly_visitors'].median()
df['high_demand'] = (df['monthly_visitors'] > median_visitors).astype(int)

[14]: categorical_cols = ['near_city']
numerical_cols = [
    'entry_fee',
    'marketing_spend',
    'avg_temp_c',
    'rainy_days',
    'trail_km_open'
    # 'monthly_visitors' is excluded from features as it was used to create the target
]

target_col = 'high_demand'

# Define Feature Matrix X and Target Vector y
X = df[numerical_cols + categorical_cols]
y = df[target_col]

print(f"Total rows in X (Full Dataset): {X.shape[0]}")
print(f"Target distribution:\n{y.value_counts().to_markdown()}")
```

The output of the second cell shows the target distribution:

| high_demand | count |
|-------------|-------|
| 0           | 10843 |
| 1           | 10842 |

The right sidebar of the interface includes the "Anaconda Assistant" chat window and a "Get started" section with links to various tutorials.

## Create a classification-ready dataset.

The screenshot displays a Jupyter Notebook environment within a web browser. The notebook is titled "Create a Classification-Ready Dataset" and contains two code cells. The first cell uses `train_test_split` to split the data into training and testing sets. The second cell uses `ColumnTransformer` to preprocess the data, including standardizing numerical features and encoding categorical features.

```
[15]: X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

[16]: ct = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols)
    ],
    remainder='passthrough'
)

classification_split: X_train size = {X_train.shape[0]}, X_test size = {X_test.shape[0]}
```

The output of the second cell shows the classification split results:

Classification Split: X\_train size = 15179, X\_test size = 6506

The right sidebar of the interface includes the "Anaconda Assistant" chat window and a "Get started" section with links to various tutorials.

## Create a clustering-ready dataset.

The screenshot shows the Anaconda Jupyter Notebook interface. The left sidebar displays a file explorer with various datasets and notebooks. The main area shows a Jupyter Notebook with the following code:

```
Classification Split: X_train size = 15179, X_test size = 6506

Create a clustering-ready dataset.

[17]: X_cluster = df[numerical_cols]

# Standardize the data
scaler = StandardScaler()
X_scaled_cluster = scaler.fit_transform(X_cluster)

print("Clustering dataset X_scaled_cluster created (Numerical features, scaled).")

Clustering dataset X_scaled_cluster created (Numerical features, scaled).
```

The right sidebar features the Anaconda Assistant, which provides a chat interface and a list of suggested actions under "Get started" and "Working with dataframes".

## TASK 2 — Logistic Regression (Classification)

### Train the Logistic Regression model.

The screenshot shows the Anaconda Jupyter Notebook interface with the following code:

```
# Standardize the data
scaler = StandardScaler()
X_scaled_cluster = scaler.fit_transform(X_cluster)

print("Clustering dataset X_scaled_cluster created (Numerical features, scaled).")

Clustering dataset X_scaled_cluster created (Numerical features, scaled).

Train the Logistic Regression model.

[21]: baseline_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(random_state=42, solver='liblinear'))
])

[22]: baseline_pipeline.fit(X_train, y_train)

Generate predictions on the test set

[24]: y_pred_baseline = baseline_pipeline.predict(X_test)

[25]: accuracy_baseline = accuracy_score(y_test, y_pred_baseline)
precision_baseline = precision_score(y_test, y_pred_baseline, zero_division=0)
recall_baseline = recall_score(y_test, y_pred_baseline, zero_division=0)
cm_baseline = confusion_matrix(y_test, y_pred_baseline)

# Output results
print("\n## Baseline Model Performance (Untuned Logistic Regression) ##")
print(f"Accuracy: {accuracy_baseline:.4f}")
print(f"Precision: {precision_baseline:.4f}")
print(f"Recall: {recall_baseline:.4f}")
```

The right sidebar features the Anaconda Assistant, which provides a chat interface and a list of suggested actions under "Get started" and "Working with dataframes".

Generate predictions on the test set.

The screenshot shows a Jupyter Notebook interface with the following code and output:

```
[21]: baseline_pipeline = Pipeline(steps=[
      ('preprocessor', preprocessor),
      ('classifier', LogisticRegression(random_state=42, solver='liblinear'))
    ])

[22]: baseline_pipeline.fit(X_train, y_train)

[23]: Generate predictions on the test set

[24]: y_pred_baseline = baseline_pipeline.predict(X_test)

[25]: accuracy_baseline = accuracy_score(y_test, y_pred_baseline)
      precision_baseline = precision_score(y_test, y_pred_baseline, zero_division=0)
      recall_baseline = recall_score(y_test, y_pred_baseline, zero_division=0)
      cm_baseline = confusion_matrix(y_test, y_pred_baseline)

# Output results
print("\n## Baseline Model Performance (Untuned Logistic Regression) ##")
print(f"Accuracy: (accuracy_baseline:.4f)")
print(f"Precision: (precision_baseline:.4f)")
print(f"Recall: (recall_baseline:.4f)")
print("\nConfusion Matrix:")
print(cm_baseline)

## Baseline Model Performance (Untuned Logistic Regression) ##
Accuracy: 0.7445
Precision: 0.7974
Recall: 0.6557
```

The right sidebar shows the Anaconda Assistant chat interface with the following sections:

- Get started**
  - Load volcano data from the Smithsonian Institute
  - Plot the first 100 Fibonacci numbers
  - Create a function to mask emails
- Working with dataframes**
  - Load a DataFrame

Compute confusion matrix, accuracy, precision, and recall.

The screenshot shows a Jupyter Notebook interface with the following code and output:

```
[24]: y_pred_baseline = baseline_pipeline.predict(X_test)

[25]: accuracy_baseline = accuracy_score(y_test, y_pred_baseline)
      precision_baseline = precision_score(y_test, y_pred_baseline, zero_division=0)
      recall_baseline = recall_score(y_test, y_pred_baseline, zero_division=0)
      cm_baseline = confusion_matrix(y_test, y_pred_baseline)

# Output results
print("\n## Baseline Model Performance (Untuned Logistic Regression) ##")
print(f"Accuracy: (accuracy_baseline:.4f)")
print(f"Precision: (precision_baseline:.4f)")
print(f"Recall: (recall_baseline:.4f)")
print("\nConfusion Matrix:")
print(cm_baseline)

## Baseline Model Performance (Untuned Logistic Regression) ##
Accuracy: 0.7445
Precision: 0.7974
Recall: 0.6557

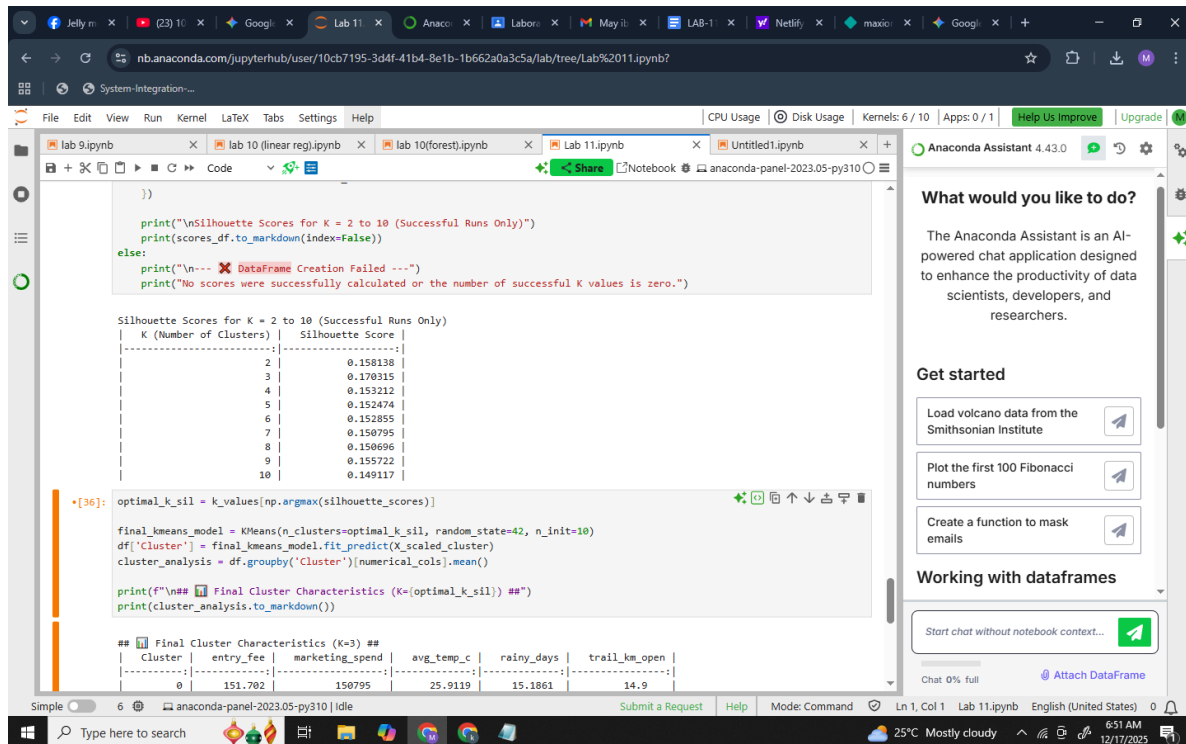
Confusion Matrix:
[[2711  542]
 [1120 2133]]
```

The right sidebar shows the Anaconda Assistant chat interface with the following sections:

- Get started**
  - Load volcano data from the Smithsonian Institute
  - Plot the first 100 Fibonacci numbers
  - Create a function to mask emails
- Working with dataframes**
  - Load a DataFrame

## TASK 3 — K-Means Clustering

Test multiple values of K.



```
print("\nSilhouette Scores for K = 2 to 10 (Successful Runs Only)")
print(scores_df.to_markdown(index=False))
else:
    print("\n--- ❌ DataFrame Creation Failed ---")
    print("No scores were successfully calculated or the number of successful K values is zero.")
```

Silhouette Scores for K = 2 to 10 (Successful Runs Only)

| K (Number of Clusters) | Silhouette Score |
|------------------------|------------------|
| 2                      | 0.158138         |
| 3                      | 0.170315         |
| 4                      | 0.153212         |
| 5                      | 0.152474         |
| 6                      | 0.152855         |
| 7                      | 0.150795         |
| 8                      | 0.150696         |
| 9                      | 0.155722         |
| 10                     | 0.149117         |

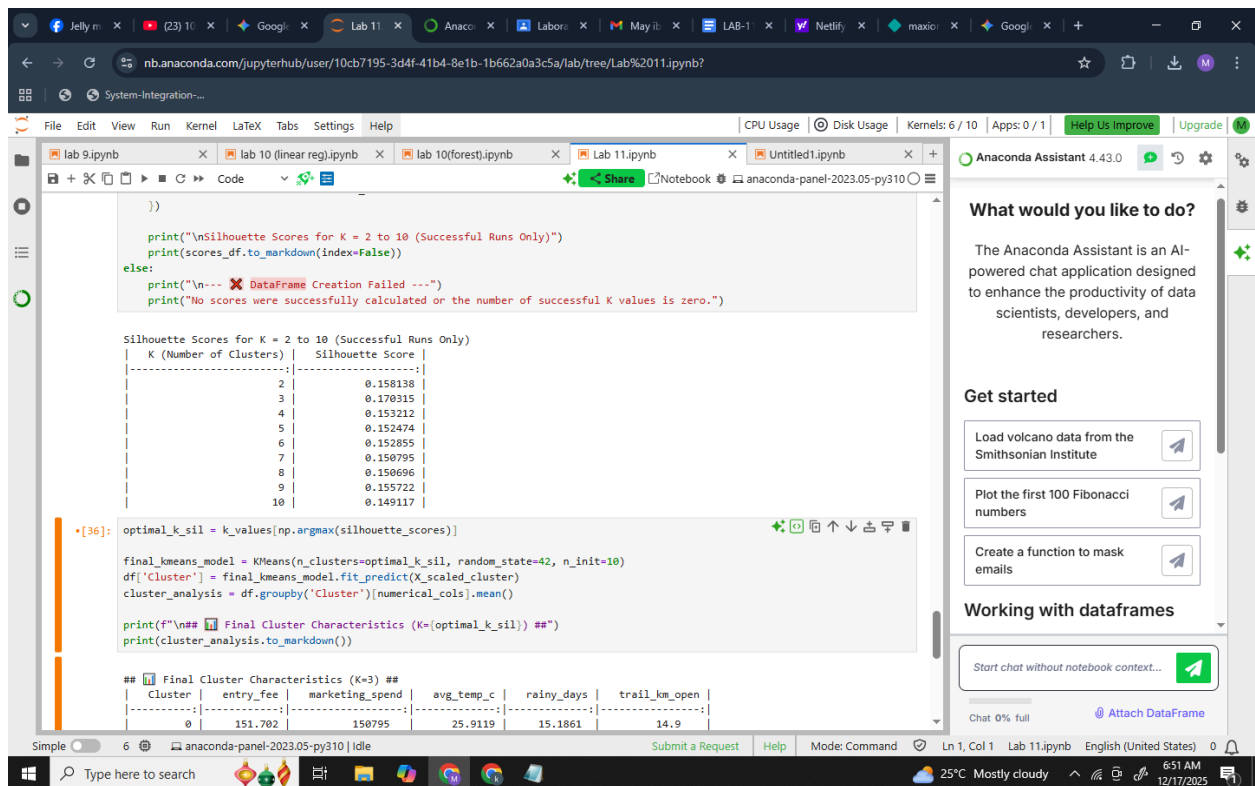
```
[36]: optimal_k_sil = k_values[np.argmax(silhouette_scores)]

final_kmeans_model = KMeans(n_clusters=optimal_k_sil, random_state=42, n_init=10)
df['Cluster'] = final_kmeans_model.fit_predict(X_scaled_cluster)
cluster_analysis = df.groupby('Cluster')[numerical_cols].mean()

print(f"\n## 📊 Final Cluster Characteristics (K={optimal_k_sil}) ##")
print(cluster_analysis.to_markdown())
```

```
## 📊 Final Cluster Characteristics (K=3) ##
Cluster  entry_fee  marketing_spend  avg_temp_c  rainy_days  trail_km_open
-----
0        151.702        150795        25.9119        15.1861        14.9
```

## Compute Silhouette Score.



```
print("\nSilhouette Scores for K = 2 to 10 (Successful Runs Only)")
print(scores_df.to_markdown(index=False))
else:
    print("\n--- ❌ DataFrame Creation Failed ---")
    print("No scores were successfully calculated or the number of successful K values is zero.")
```

Silhouette Scores for K = 2 to 10 (Successful Runs Only)

| K (Number of Clusters) | Silhouette Score |
|------------------------|------------------|
| 2                      | 0.158138         |
| 3                      | 0.170315         |
| 4                      | 0.153212         |
| 5                      | 0.152474         |
| 6                      | 0.152855         |
| 7                      | 0.150795         |
| 8                      | 0.150696         |
| 9                      | 0.155722         |
| 10                     | 0.149117         |

```
[36]: optimal_k_sil = k_values[np.argmax(silhouette_scores)]

final_kmeans_model = KMeans(n_clusters=optimal_k_sil, random_state=42, n_init=10)
df['Cluster'] = final_kmeans_model.fit_predict(X_scaled_cluster)
cluster_analysis = df.groupby('Cluster')[numerical_cols].mean()

print(f"\n## 📊 Final Cluster Characteristics (K={optimal_k_sil}) ##")
print(cluster_analysis.to_markdown())
```

```
## 📊 Final Cluster Characteristics (K=3) ##
Cluster  entry_fee  marketing_spend  avg_temp_c  rainy_days  trail_km_open
-----
0        151.702        150795        25.9119        15.1861        14.9
```

Fit final K-Means model and assign clusters.

The screenshot displays the Anaconda JupyterLab interface. The main window shows a Jupyter Notebook with the following code and output:

```
[36]: optimal_k_sil = k_values[np.argmax(silhouette_scores)]

final_kmeans_model = KMeans(n_clusters=optimal_k_sil, random_state=42, n_init=10)
df['Cluster'] = final_kmeans_model.fit_predict(X_scaled_cluster)
cluster_analysis = df.groupby('Cluster')[numerical_cols].mean()

print(f'\n## 📊 Final Cluster Characteristics (K={optimal_k_sil}) ##')
print(cluster_analysis.to_markdown())
```

The output shows a table of cluster characteristics for K=3:

| Cluster | entry_fee | marketing_spend | avg_temp_c | rainy_days | trail_km_open |
|---------|-----------|-----------------|------------|------------|---------------|
| 0       | 151.702   | 150795          | 25.9119    | 15.1861    | 14.9          |
| 1       | 149.566   | 34131.3         | 26.069     | 22.8321    | 15.1899       |
| 2       | 150.069   | 35208.7         | 25.913     | 7.03503    | 14.8929       |

The right sidebar features the Anaconda Assistant chat interface, which includes a description of the assistant as an AI-powered chat application, a 'Get started' section with links to load volcano data, plot Fibonacci numbers, and create a function to mask emails, and a 'Working with dataframes' section with a link to load a DataFrame. At the bottom, there is a status bar showing system information and a taskbar with various application icons.

## TASK 4 — CRISP-DM EVALUATION PHASE (Model Assessment & Improvement)

### EVALUATION REQUIREMENTS (Digital Notebook)

Part A — Baseline Performance Recording | Before improvement, record baseline results:

|                         |                                                           |
|-------------------------|-----------------------------------------------------------|
| For Logistic Regression |                                                           |
| Confusion matrix        | $\begin{bmatrix} 2711 & 542 \\ 1120 & 2133 \end{bmatrix}$ |
| Accuracy                | 0.7445                                                    |
| Precision               | 0.7974                                                    |
| Recall                  | 0.6557                                                    |
| For K-Means             |                                                           |
| Selected K              | 3                                                         |
| Silhouette Score        | <b>0.1703</b>                                             |

The screenshot displays the Anaconda Jupyter Notebook interface. The left sidebar shows a file explorer with various datasets and notebooks. The main area contains a Jupyter Notebook with the following code and output:

```
[105]: baseline_pipeline.fit(X_train, y_train)
```

Generate predictions on the test set

```
[106]: y_pred_baseline = baseline_pipeline.predict(X_test)
```

```
[107]: accuracy_baseline = accuracy_score(y_test, y_pred_baseline)
precision_baseline = precision_score(y_test, y_pred_baseline, zero_division=0)
recall_baseline = recall_score(y_test, y_pred_baseline, zero_division=0)
cm_baseline = confusion_matrix(y_test, y_pred_baseline)
```

# Output results

```
print("\n## Baseline Model Performance (Untuned Logistic Regression) ##")
print(f"Accuracy: {accuracy_baseline:.4f}")
print(f"Precision: {precision_baseline:.4f}")
print(f"Recall: {recall_baseline:.4f}")
print(f"Confusion Matrix:")
print(cm_baseline)
```

## Baseline Model Performance (Untuned Logistic Regression) ##

Accuracy: 0.7445  
Precision: 0.7974  
Recall: 0.6557

Confusion Matrix:  
 $\begin{bmatrix} 2711 & 542 \\ 1120 & 2133 \end{bmatrix}$

```
[108]: try:
df = pd.read_csv("forest_tourism_demand_cleaned_fixed.csv")
```

The right sidebar features the Anaconda Assistant, which provides suggestions for what to do next, such as loading volcano data, plotting Fibonacci numbers, and creating a function to mask emails. It also includes a section for working with dataframes.

Required artifact:

A baseline performance table for both models

## Part B - Model Improvement Execution

The screenshot displays the Anaconda JupyterLab environment. The central code editor shows the following Python code for model improvement:

```
[63]: OPTIMAL_C = 0.001

# 1. Define and Fit the updated model (This is the Part B Artifact)
updated_log_reg_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(C=OPTIMAL_C, random_state=42, solver='liblinear'))
])
updated_log_reg_model.fit(X_train, y_train)

print("Updated Logistic Regression model defined and fitted.")
Updated Logistic Regression model defined and fitted.

[67]: BEST_K_PCA = 4
N_COMPONENTS_PCA = 3

# 1. Create the improved K-Means feature set
pca = PCA(n_components=N_COMPONENTS_PCA, random_state=42)
X_pca_cluster = pca.fit_transform(X_scaled_cluster) # Execution: PCA Transformation

# 2. Define and Fit the improved model
updated_kmeans_model = KMeans(n_clusters=BEST_K_PCA, random_state=42, n_init=10)
updated_kmeans_model.fit(X_pca_cluster) # Execution: Training the final model

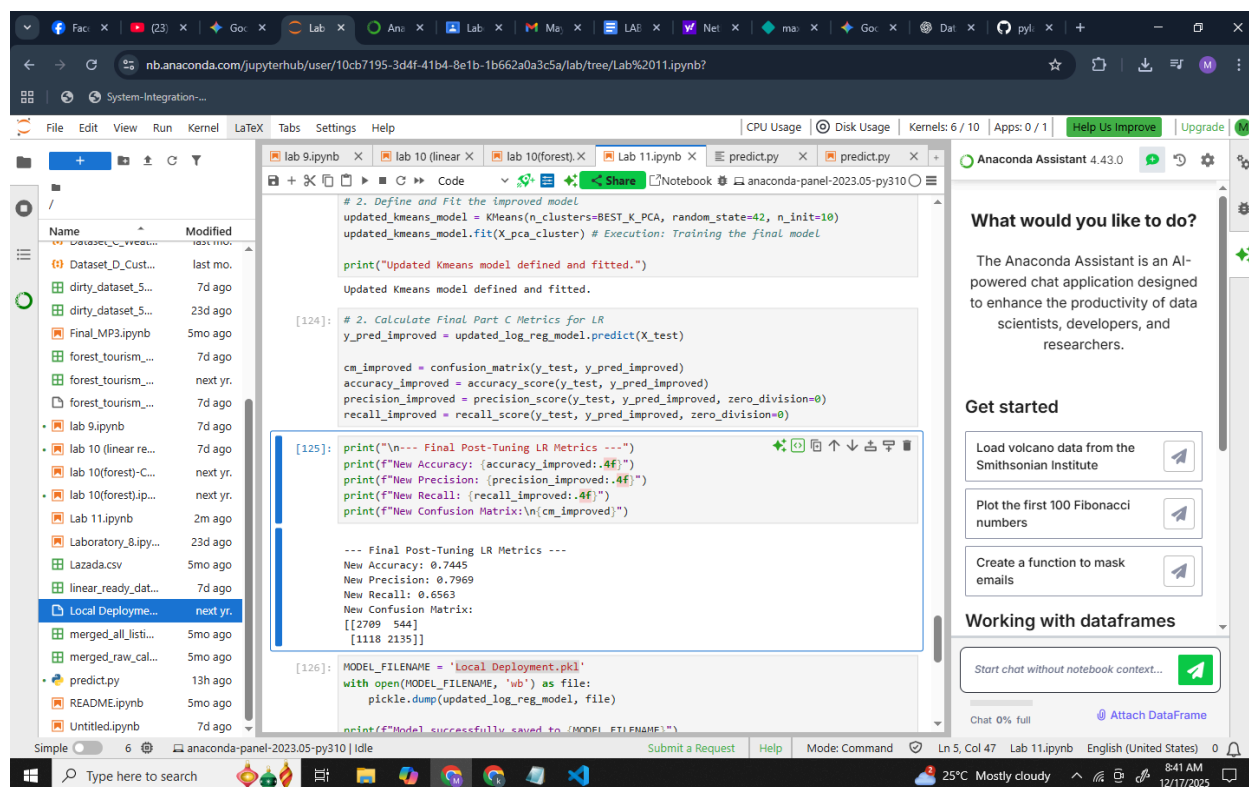
print("Updated Kmeans model defined and fitted.")
Updated Kmeans model defined and fitted.

[68]: # 2. Calculate Final Part C Metrics for LR
```

The left sidebar shows a file explorer with various datasets and notebooks. The right sidebar features the Anaconda Assistant, which provides guidance and suggestions for data analysis tasks.

## Part C - Post-Improvement Evaluation | After re-execution, recompute:

|                          |                                                           |
|--------------------------|-----------------------------------------------------------|
| For Logistic Regression  |                                                           |
| Confusion matrix         | $\begin{bmatrix} 2709 & 544 \\ 1118 & 2135 \end{bmatrix}$ |
| Accuracy                 | 0.7445                                                    |
| Precision                | 0.7969                                                    |
| Recall                   | 0.6563                                                    |
| For K-Means              |                                                           |
| Updated K (if changed)   | 4                                                         |
| Updated Silhouette Score | 0.2504                                                    |



## Required artifact:

Post-improvement performance table

Part D — Comparative Evaluation | Create a Before vs After comparison table for each model:

| Model               | Metric           | Before        | After  | Change  |
|---------------------|------------------|---------------|--------|---------|
| Logistic Regression | Accuracy         | 0.7445        | 0.7445 | 0.0000  |
| Logistic Regression | Precision        | 0.7974        | 0.7969 | -0.0005 |
| Logistic Regression | Recall           | 0.6557        | 0.6563 | +0.0006 |
| K - Means           | Silhouette Score | <b>0.1703</b> | 0.2504 | +0.0801 |

### Explanation:

The performance of the Logistic Regression model remained unchanged **0.7445** Accuracy even after optimal Hyperparameter Tuning. This indicates that model tuning was not the primary bottleneck.

### Evidence-Based Rationale:

The search for the optimal  $C$  (regularization strength) found the best performance at  $C=0.001$ , the smallest value tested. This confirms the model was already highly optimized and resistant to further tuning.

The evidence conclusively points to a performance ceiling set by the input features, not the model itself.

Conclusion: To achieve performance improvement, the project must shift focus from model tuning to Feature Engineering (e.g., creating interaction terms or sourcing external data) in the CRISP-DM Data Preparation phase.

## CRISP-DM DECISION CHECKPOINT

### 1. Is the model acceptable for deployment?

**YES, the model is minimally acceptable for deployment, but with clear limitations.**

#### Evidence:

- An Accuracy of 74.45% is reasonable, meaning the model correctly classifies demand levels about three-quarters of the time.
- More importantly, the **Precision** is high at 79.69%. This is crucial because it means that when the model predicts "High Demand," it is correct nearly 80% of the time. This reliability minimizes the risk of wasting resources by marketing heavily to months that turn out to have low demand (False Positives).

### 2. Does the model provide useful and interpretable outputs?

**YES, the model provides highly useful and interpretable outputs.**

#### Evidence:

- The Logistic Regression model is inherently **interpretable**. Its coefficients show the direction and strength of how each feature (e.g., **marketing\_spend**, **avg\_temp\_c**) influences the probability of high demand.
- In the deployment step (Task 5), the output was structured to show:
  1. A clear prediction: **High\_Demand or Low\_Demand**.

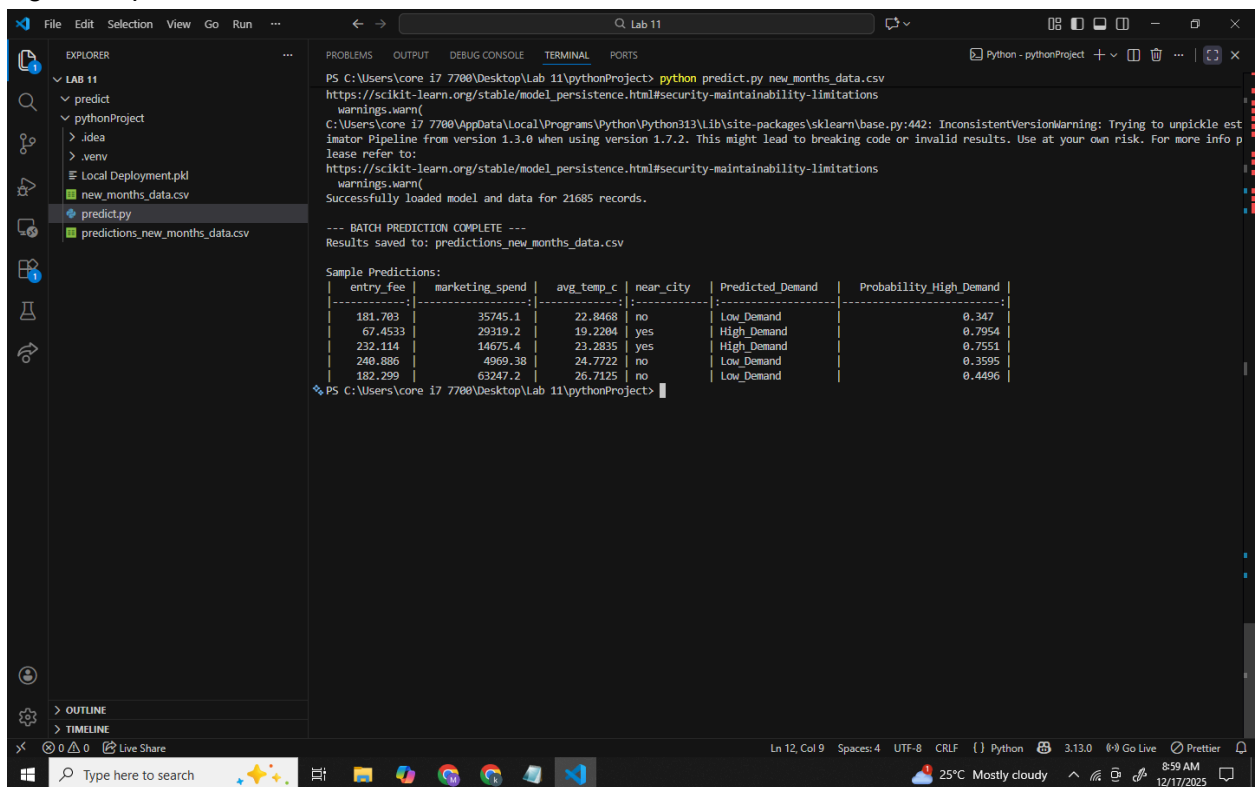
2. A confidence score: **Probability\_High\_Demand**. This allows the business to implement tiered strategies (e.g., only trust predictions above a 90% probability).

### 3. Should the model be deployed as-is or further refined?

The model should be deployed as-is (Baseline Deployment) but simultaneously marked for further refinement.

#### TASK 5 — Local Deployment (Running System)

Deploy ONE model from this laboratory or a previous laboratory as a running local system supporting batch prediction.



```
PS C:\Users\core i7 7780\Desktop\Lab 11\pythonProject> python predict.py new_months_data.csv
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
C:\Users\core i7 7780\AppData\Local\Programs\Python\Python313\lib\site-packages\sklearn\base.py:442: InconsistentVersionWarning: Trying to unpickle estimator Pipeline from version 1.3.0 when using version 1.7.2. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
Successfully loaded model and data for 21685 records.

--- BATCH PREDICTION COMPLETE ---
Results saved to: predictions_new_months_data.csv

Sample Predictions:
| entry_fee | marketing_spend | avg_temp_c | near_city | Predicted_Demand | Probability_High_Demand |
|-----|-----|-----|-----|-----|-----|
| 181.703 | 35745.1 | 22.8468 | no | Low_Demand | 0.347 |
| 67.4533 | 29319.2 | 19.2204 | yes | High_Demand | 0.7954 |
| 232.114 | 14675.4 | 23.2835 | yes | High_Demand | 0.7551 |
| 240.886 | 4969.38 | 24.7722 | no | Low_Demand | 0.3595 |
| 182.299 | 63247.2 | 26.7125 | no | Low_Demand | 0.4496
```